Học viện Công nghệ Bưu chính Viễn thông
Khoa Công nghệ Thông tin 1

Nhập môn học sâu

Mạng nơ ron hồi tiếp

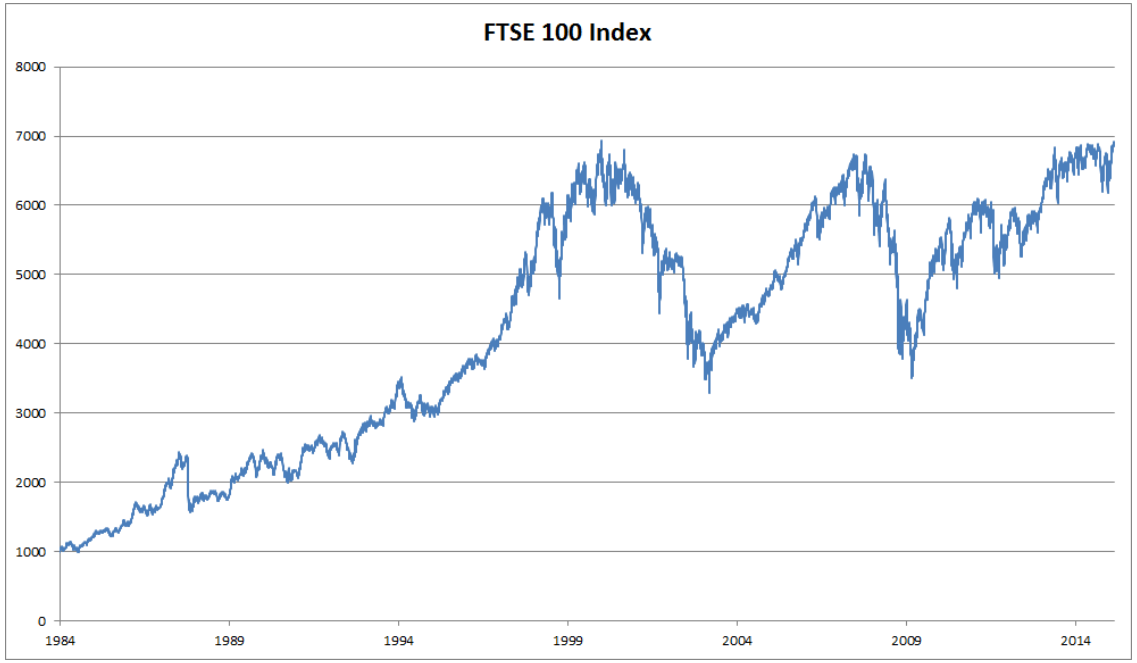
TS. NGUYỄN XUÂN ĐỨC

Nội dung

- Mô hình ngôn ngữ
- Giới thiệu mạng hồi tiếp
- Biến thể của mạng hồi tiếp
- Ứng dụng mạng hồi tiếp

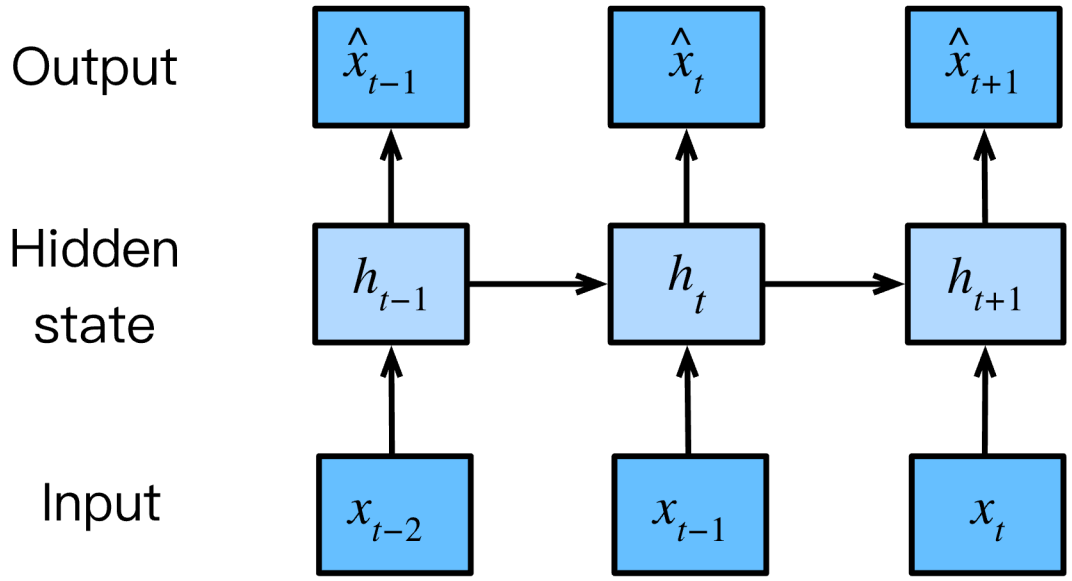
Mô hình autoregressive (tự hồi quy)

- Ví dụ: giao dịch cổ phiếu
  - Bài toán: Cần xác định giá x_t tăng hay giảm
- Mô hình tự hồi quy: $P(x_t|x_{t-1}, \dots, x_1)$
  - Vấn đề: lượng dữ liệu tăng khi chúng ta quan sát thêm nhiều dữ liệu
  - Giải pháp?



Mô hình tự hồi quy ẩn (latent autoregressive)

- Mô hình tự hồi quy ẩn: ước lượng $\hat{x}_t \sim P(x_t|h_t)$ và cập nhật $h_t = g(h_{t-1}, x_{t-1})$



Mô hình chuỗi

- Ước lượng xác suất đồng thời của cả chuỗi:

$$P(x_1, \dots, x_T) = P(x_1) \prod_{t=2}^{T} P(x_t | x_{t-1}, \dots, x_1)$$

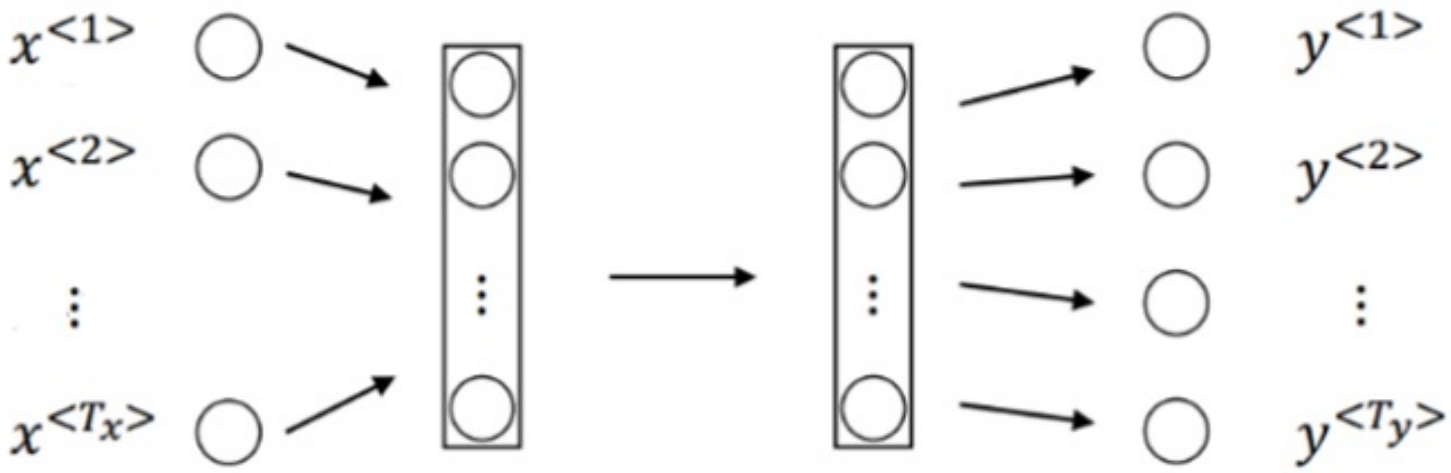
- Mô hình Markov bậc nhất:

$$P(x_1, \dots, x_T) = P(x_1) \prod_{t=2}^{T} P(x_t | x_{t-1})$$

Mô hình ngôn ngữ

Vấn đề với mạng nơ ron truyền thống

- Đầu vào và đầu ra của các mẫu huấn luyện có thể có độ dài khác nhau
- Không chia sẻ các đặc trưng học được từ các phần khác nhau của văn bản

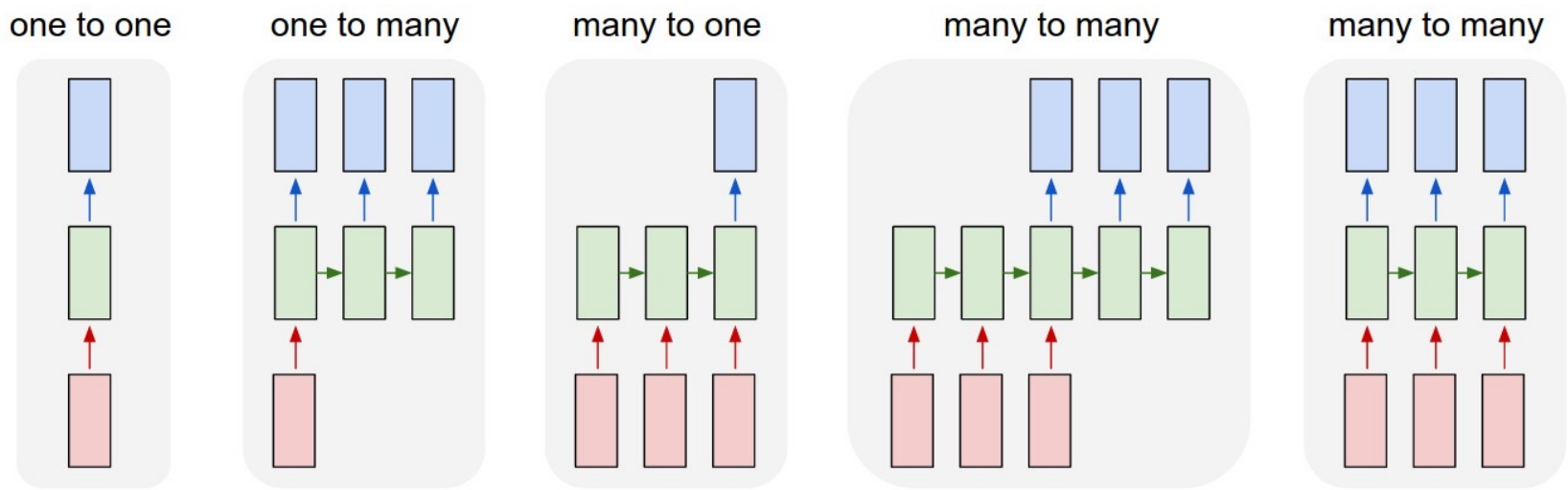


Mạng hồi tiếp (Recurrent Neural Network)

- Mạng hồi tiếp (Recurrent Neural Network) là một dạng mạng nơ ron thường được sử dung cho dữ lieu dạng chuỗi hoặc time series
- Ứng dụng:
  - Xử lý ngôn ngữ tự nhiên
  - Dịch máy
  - Nhận diện giọng nói
  - Chú thích ảnh

Mạng hồi tiếp

- Mạng hồi tiếp (Recurrent Neural Networks) thường được sử dụng cho bài toán xử lý chuỗi
- RNNs có thể làm việc với đầu vào và/hoặc đầu ra là một chuỗi



Ví dụ bài toán RNN

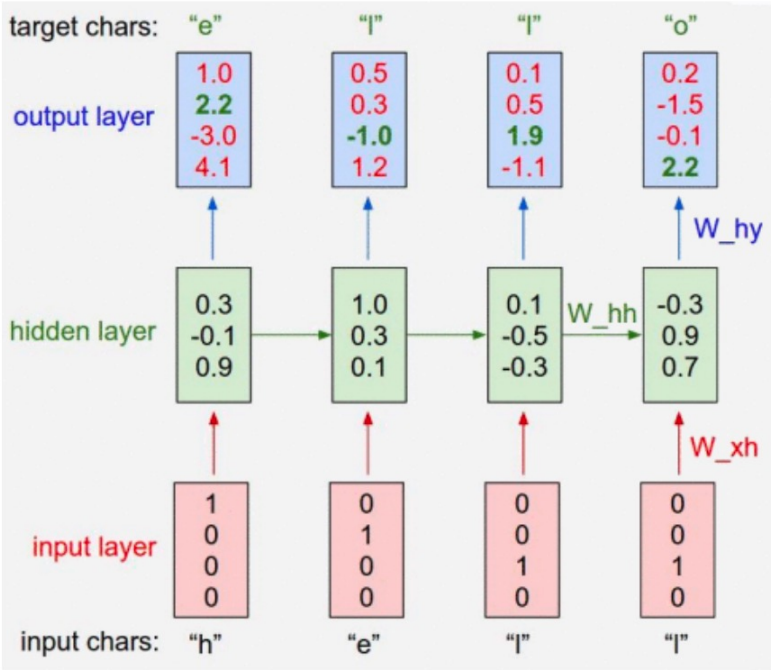
- Yêu cầu: RNN nhận đầu vào là một dãy các chữ cái và đoán chữ cái tiếp theo. Đầu ra là xác suất của mỗi chữ cái trong từ vựng:
  - Từ vựng: [h, l, e, o]
  - Mẫu huấn luyện: "hello"
- Công thức của RNN:

$$h_t = tanh(W_{hh}h_{t-1} + W_{xh}x_t)$$

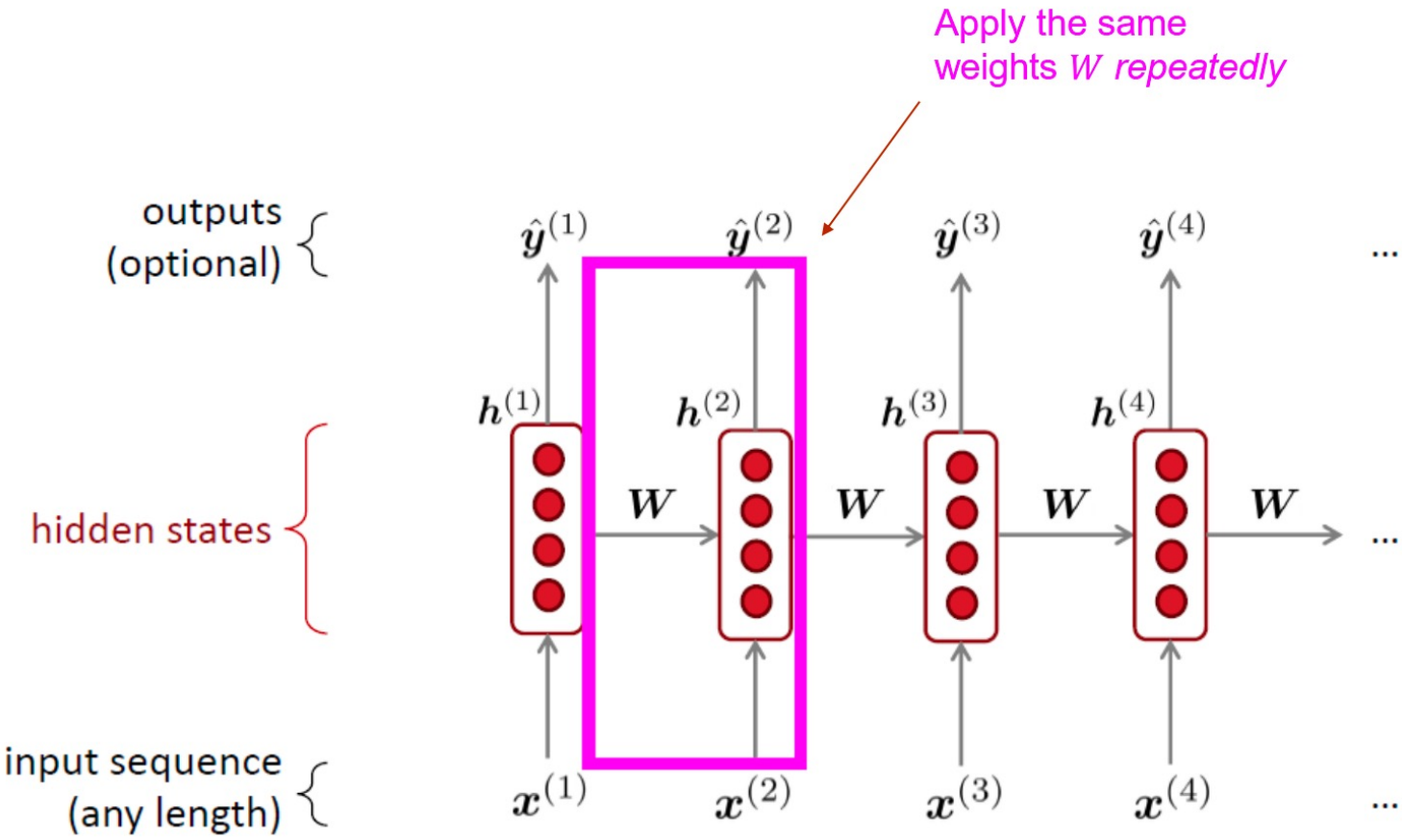
$$y_t = W_{hy}h_t$$

$$h_0 = 0$$

- Ô màu xanh ở đầu ra là hàm softmax



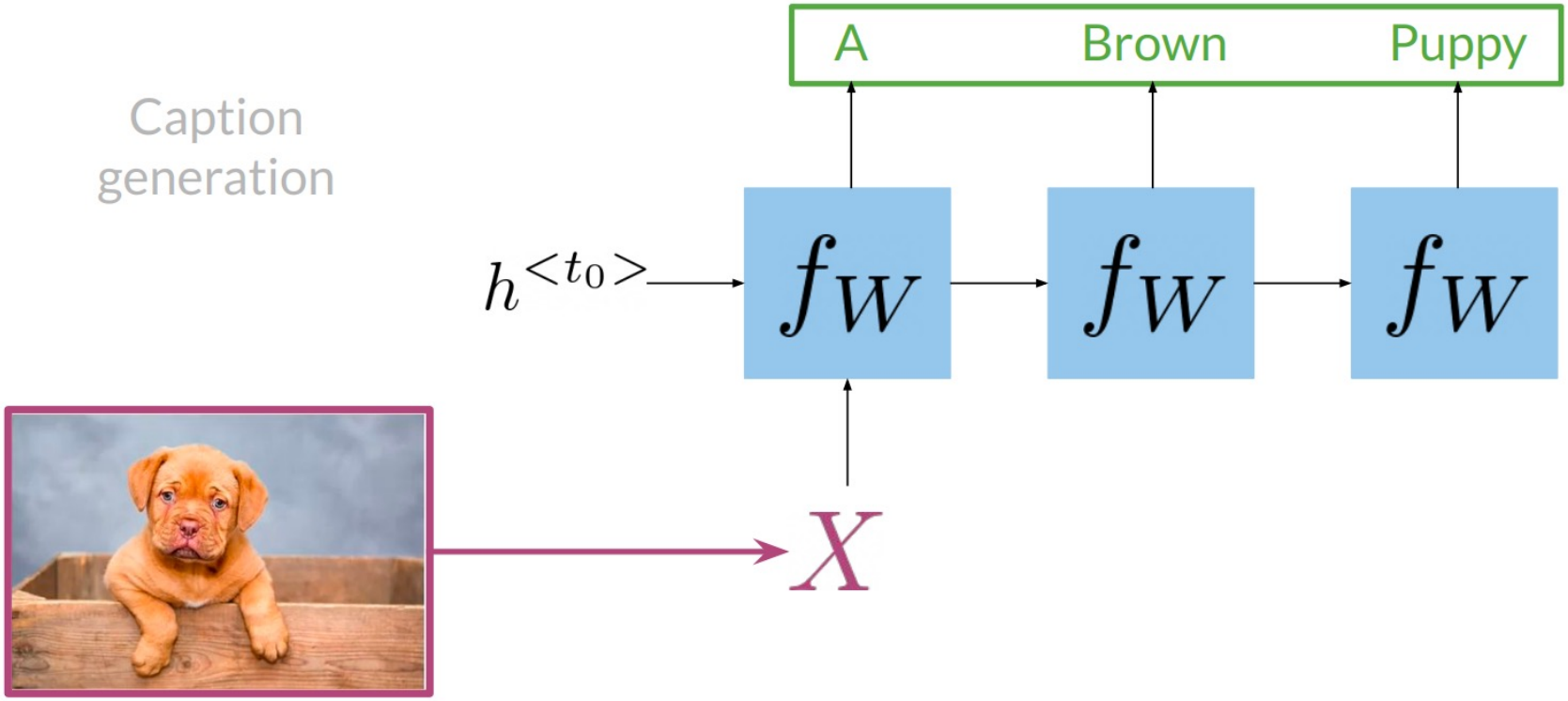
Kiến trúc mạng hồi tiếp (RNN)



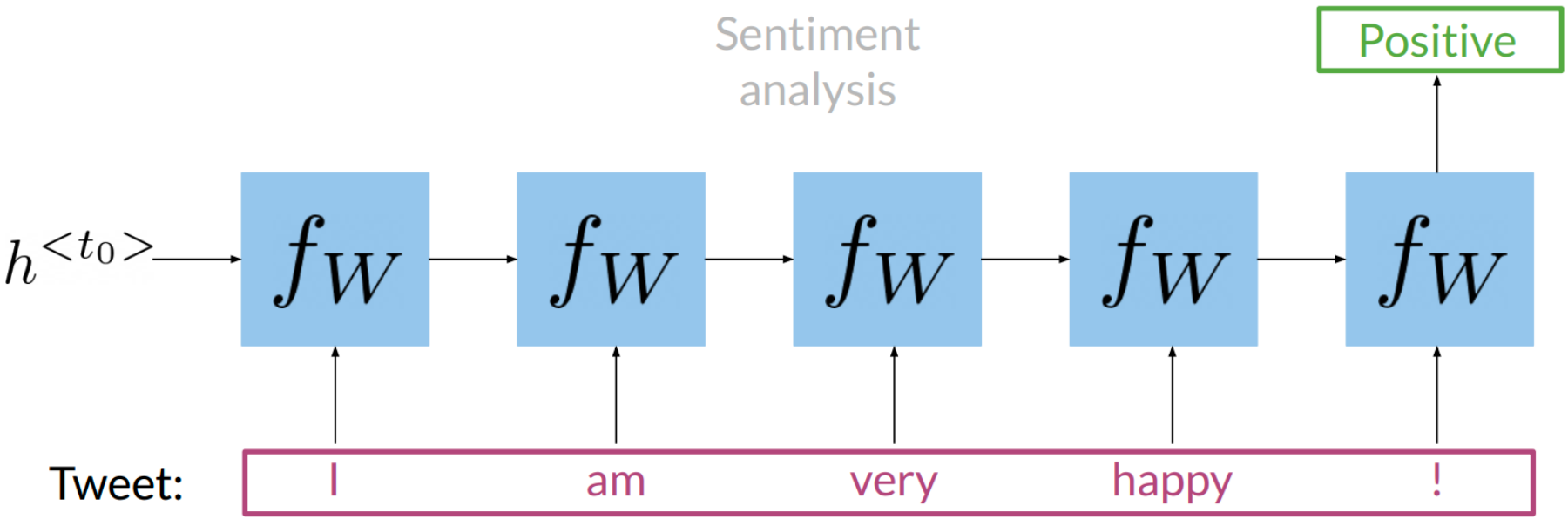
Dạng RNN – One to One

Dạng RNN – One to Many

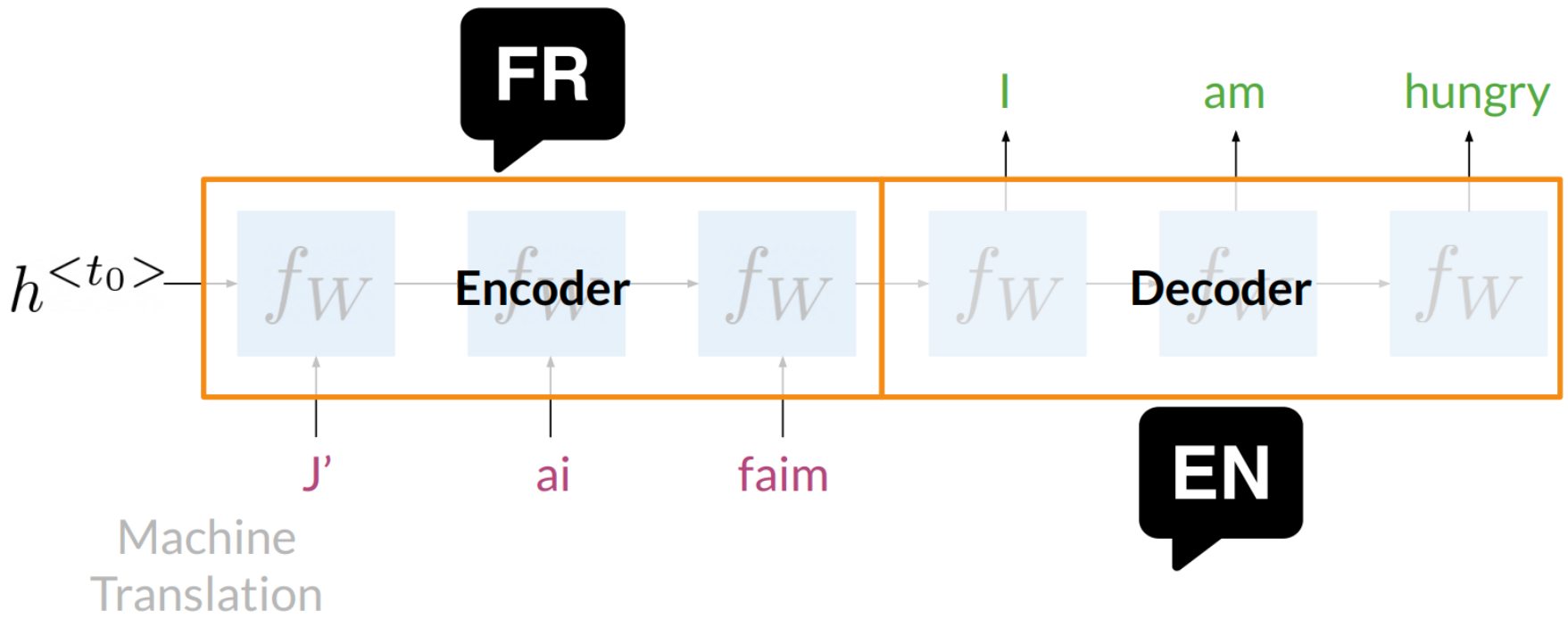
Caption generation



Dạng RNN – Many to One



Dạng RNN – Many to Many

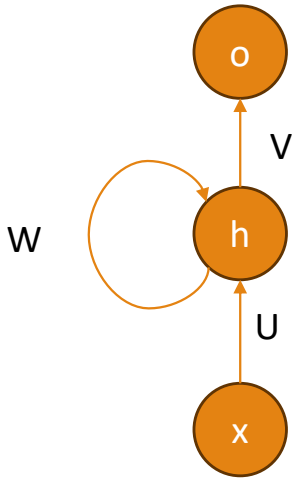


Vanilla RNN

- Vanilla RNN có ba tập tham số
  - U ánh xạ đầu vào sang trạng thái ẩn
  - W biến đổi trạng thái ẩn cũ
  - V ánh xạ trạng thái ẩn sang đầu ra
- Công thức RNN:

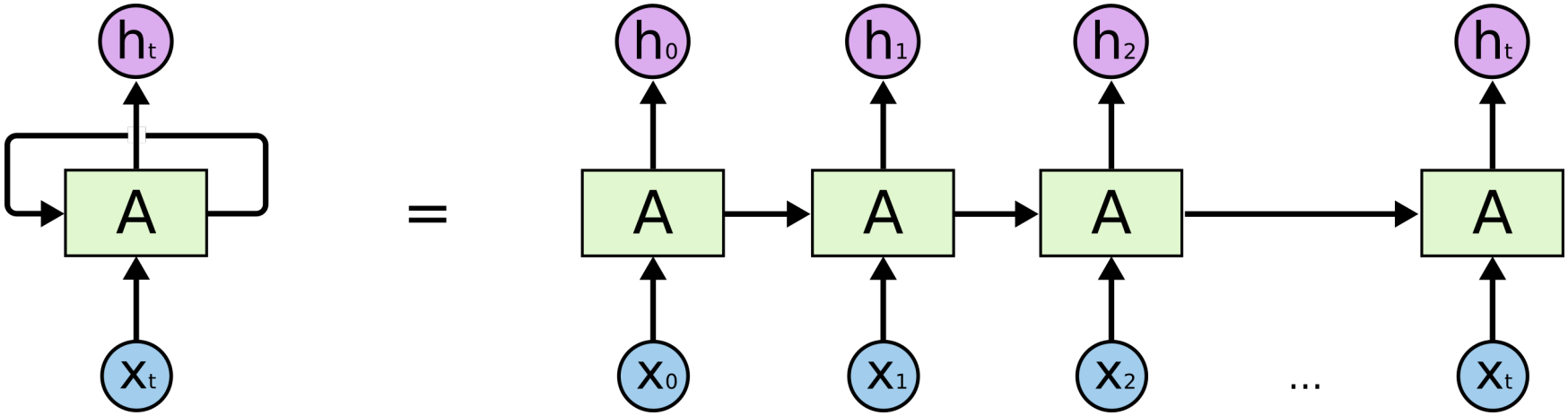
$$h_t = \phi(Wh_{t-1} + Ux_t)$$

$$o_t = Vh_t$$

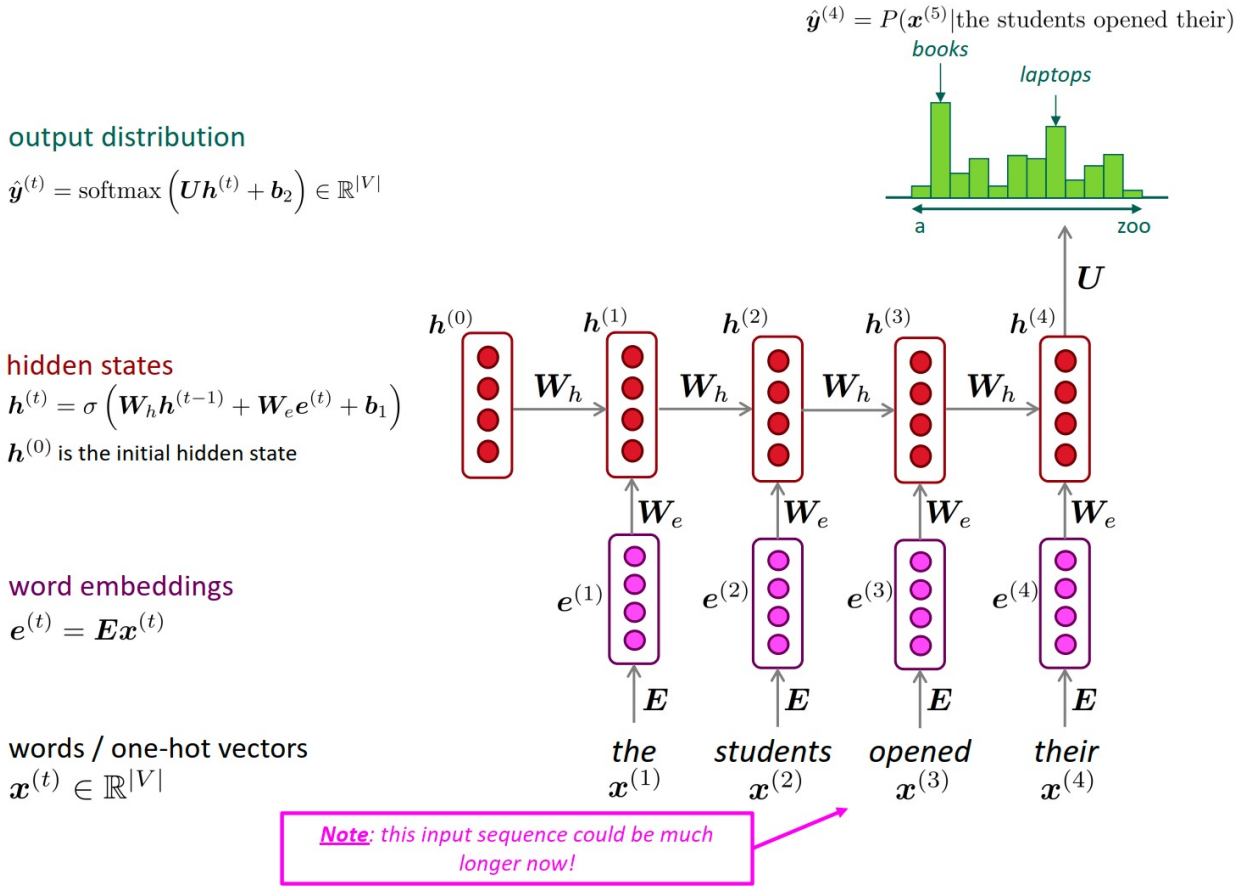


Duỗi vòng lặp RNN

- Có thể hình dung khi duỗi vòng lặp của RNN như sau:



Mô hình ngôn ngữ với RNN



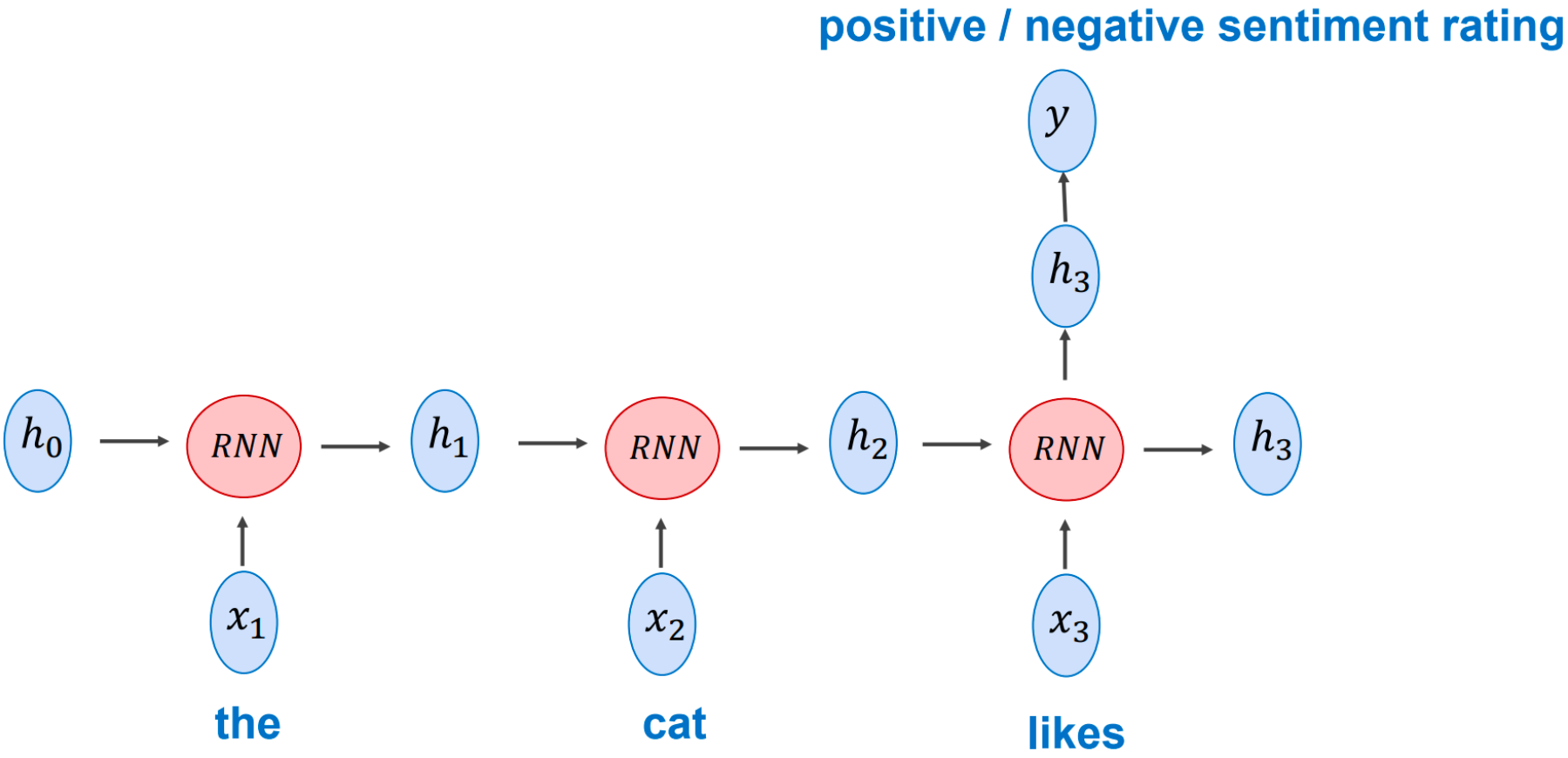
Perplexity

- Mô hình ngôn ngữ tốt: có khả năng dự đoán từ tiếp theo một cách chính xác
- Ví dụ:
  - It is raining outside
  - It is raining banana tree
  - It is raining asdf
- Mô hình ngôn ngữ tốt sẽ giúp sử dụng ít bit hơn để nén chuỗi. Chúng ta có thể dùng trung bình giá trị cross-entropy trên cả chuỗi:

$$CE = \frac{1}{n}\sum_{t=1}^{n} -logP(x_t|x_{t-1}, \dots, x_1)$$

- Perplexity: exp(CE)

RNN cho bài toán phân tích quan điểm



Mô hình ngôn ngữ với RNN

- Ưu điểm
  - Xử lý đầu vào với độ dài bất kỳ
  - Tính toán tại mỗi bước có thể sử dụng thông tin từ nhiều bước trước đó
  - Kích thước mô hình không tăng khi độ dài đầu vào tăng
  - Vì sử dụng cùng trọng số, đầu vào được xử lý tương tự nhau
- Nhược điểm
  - Tốc độ xử lý chậm do tính toán tuần tự
  - Trong thực tế, khó có thể truy cập thông tin từ **quá nhiều bước trước đó** (RNN thường hoạt động tốt với 10-20 bước)

Huấn luyện RNN (mô hình ngôn ngữ)

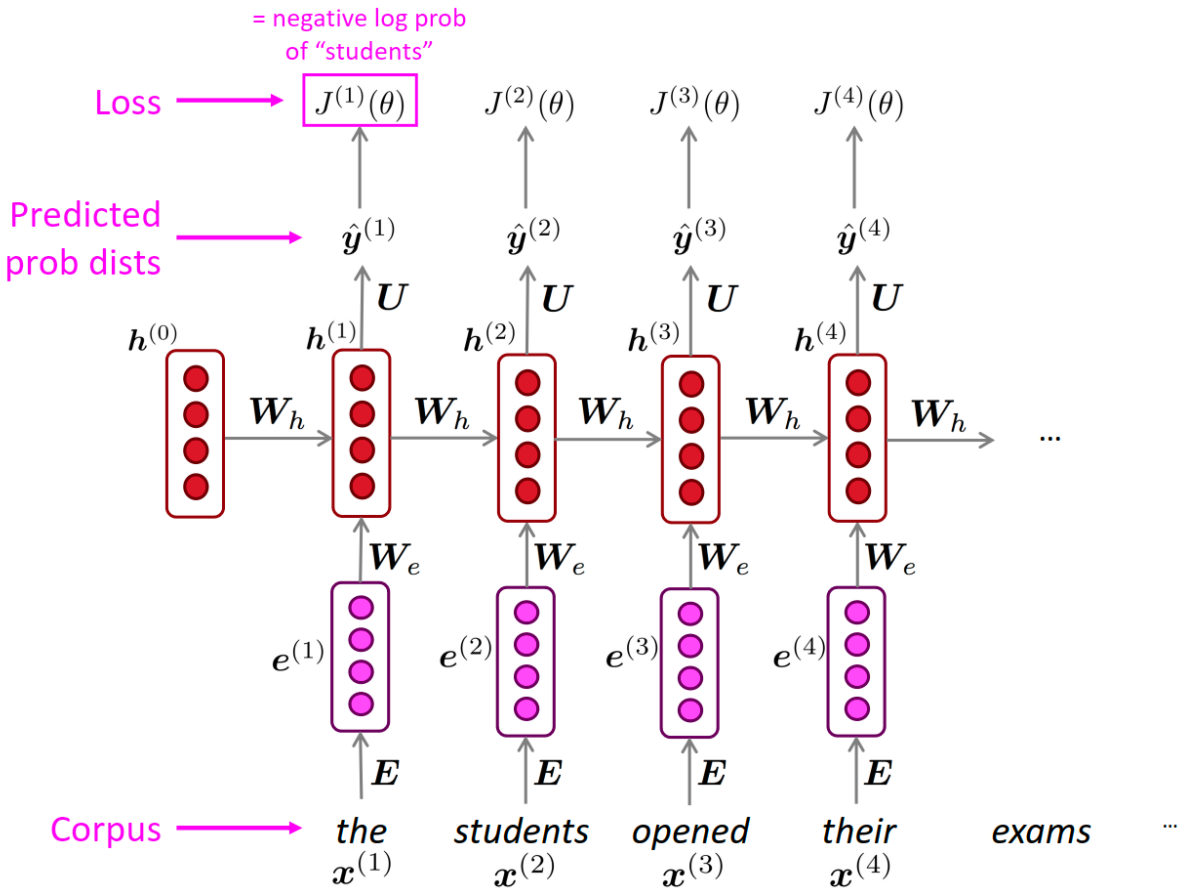
- Đầu vào: tập văn bản với mỗi câu dạng $x^{(1)}, \ldots, x^{(T)}$
- Tính phân phối đầu ra $\hat{y}^{(t)}$ với mọi bước t
  - Tính phân phối của mọi từ, với các từ đã cho
- Hàm mất mát ở bước t là **cross-entropy** giữa xác suất dự đoán $\hat{y}^{(t)}$, và từ kế tiếp $y^{(t)}$ (one-hot vector của $x^{(t+1)}$):

$$J^{(t)}(\theta) = CE\left(y^{(t)}, \hat{y}^{(t)}\right) = -\sum_{w \in V} y_w^{(t)} log \hat{y}_w^{(t)} = -log \hat{y}_{x_{t+1}}^{(t)}$$

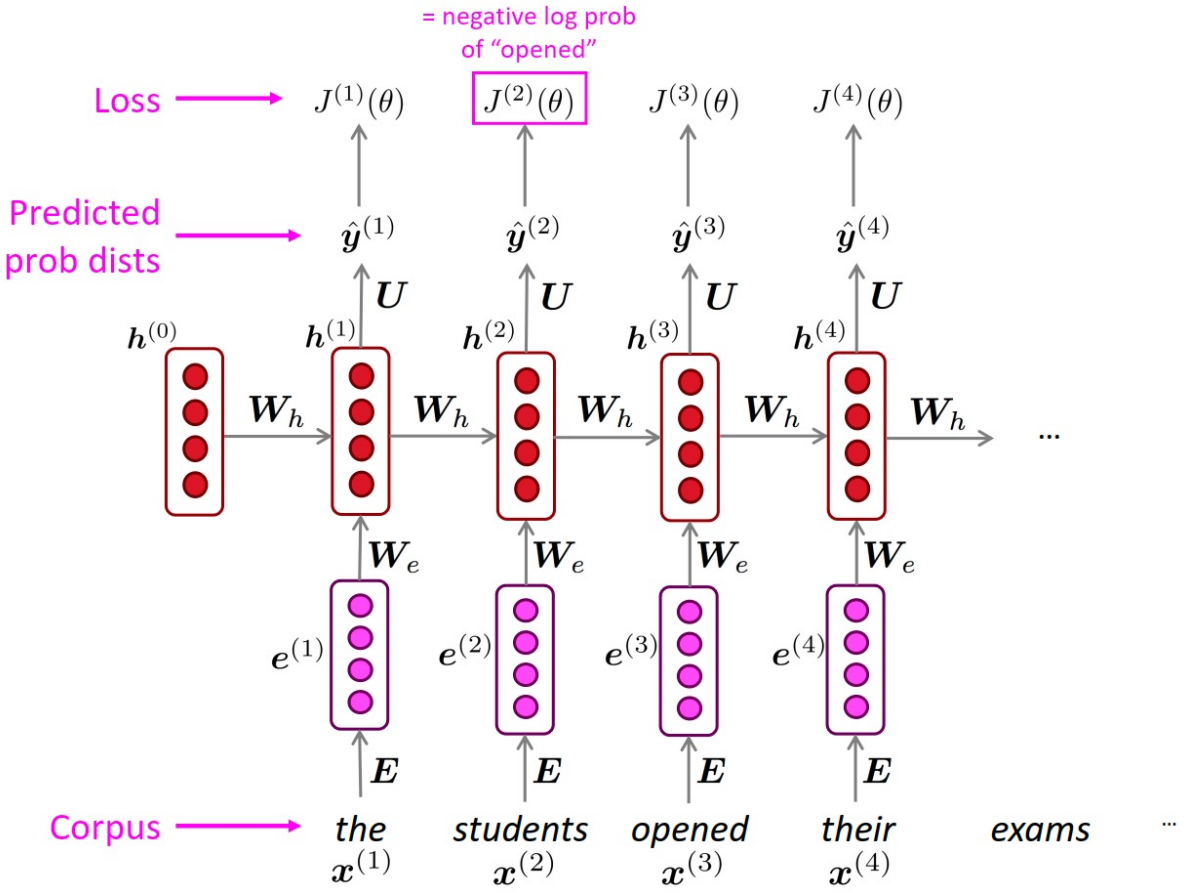
- Tính trung bình để lấy giá trị mất mát cho cả câu:

$$J(\theta) = \frac{1}{T}\sum_{t=1}^{T} J^{(t)}(\theta) = \frac{1}{T}\sum_{t=1}^{T} -log \hat{y}_{x_{t+1}}^{(t)}$$

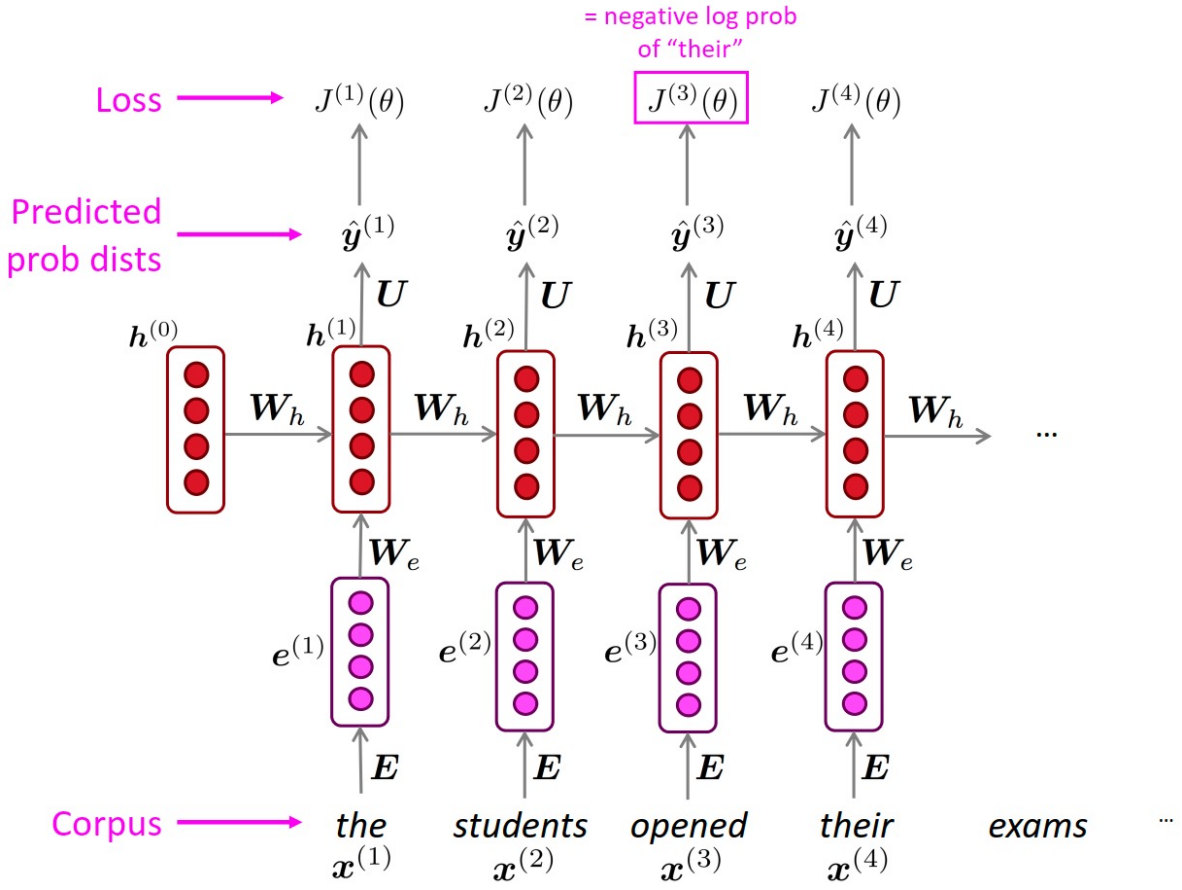
Huấn luyện RNN (mô hình ngôn ngữ)



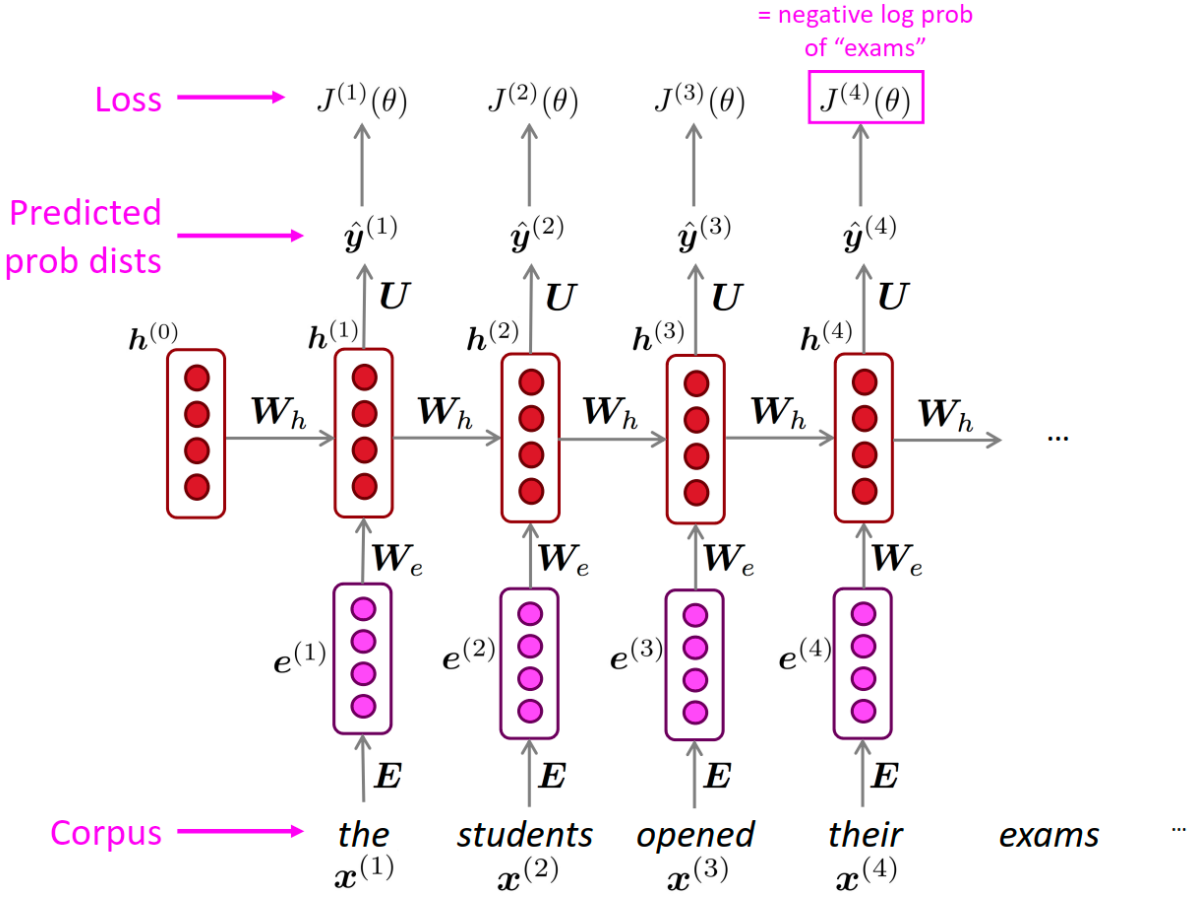
Huấn luyện RNN (mô hình ngôn ngữ)



Huấn luyện RNN (mô hình ngôn ngữ)

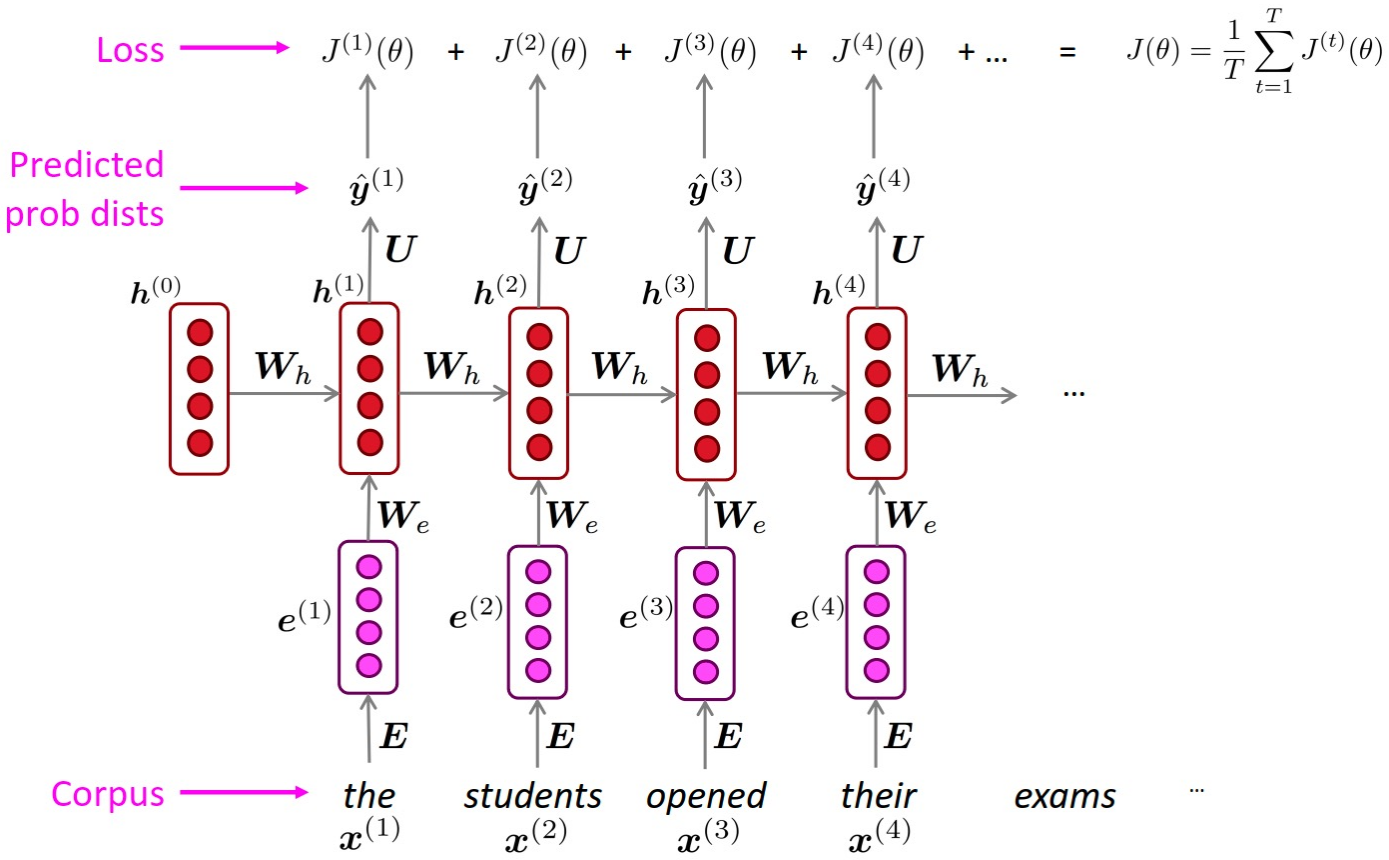


Huấn luyện RNN (mô hình ngôn ngữ)



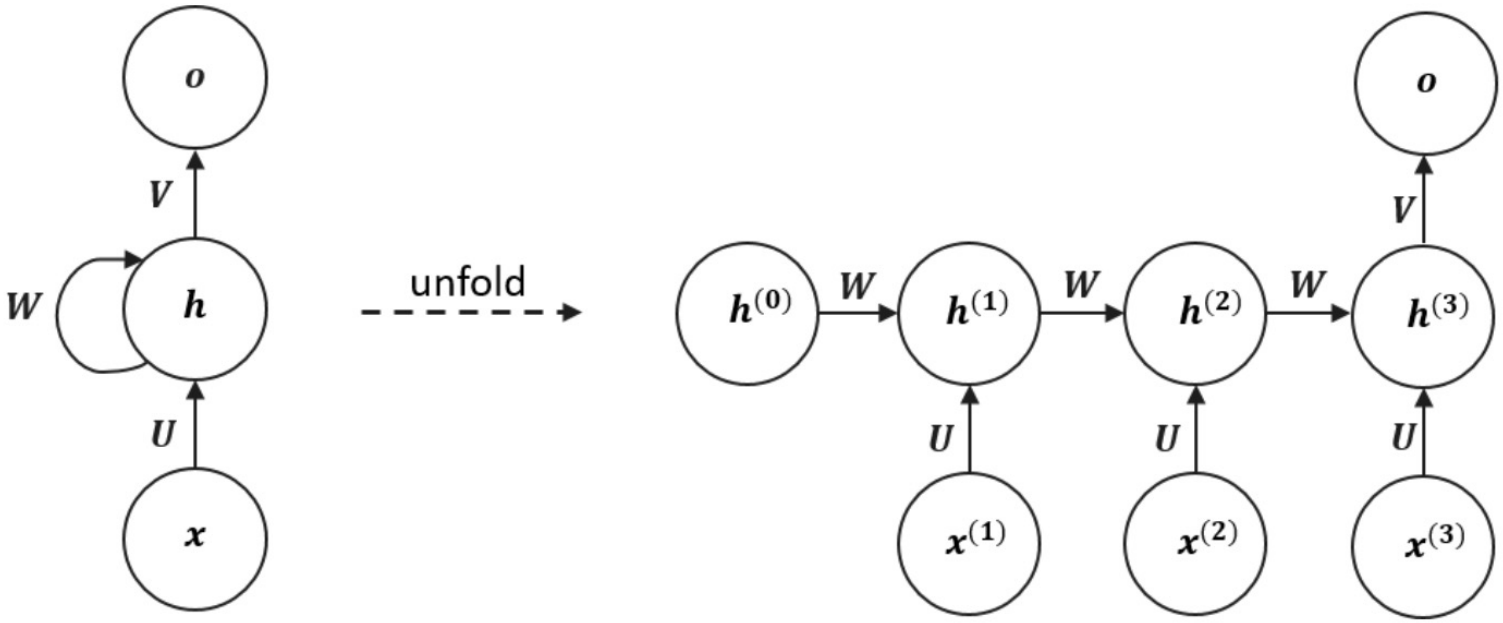
Huấn luyện RNN (mô hình ngôn ngữ)

- Teacher forcing: đưa giá trị ground truth làm đầu vào của RNN tại mỗi bước



Backprop theo thời gian

- Sau khi duỗi RNN, ta thấy gradient chảy ngược về theo thời gian nên được gọi là thuật toán backprop theo thời gian (backpropagation through time)



Backprop theo thời gian

- Với hàm mục tiêu L(y, o), ta có đạo hàm của hàm mục tiêu theo trọng số V, W, và U như sau:

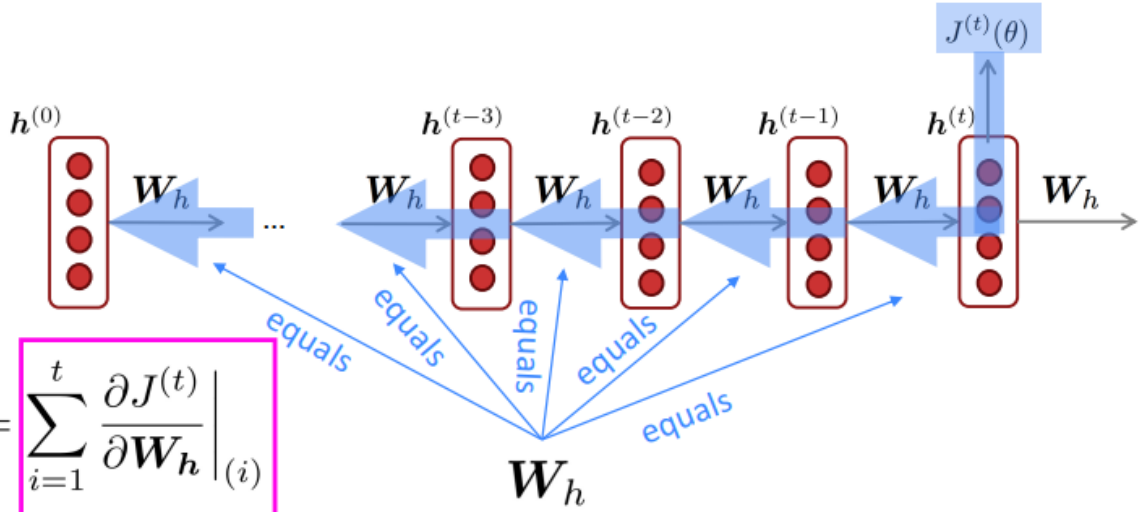
$$\frac{\partial L}{\partial \mathbf{V}} = \frac{\partial L}{\partial \mathbf{o}} \frac{\partial \mathbf{o}}{\partial \mathbf{V}}$$

$$\frac{\partial L}{\partial \mathbf{W}} = \frac{\partial L}{\partial \mathbf{o}} \frac{\partial \mathbf{o}}{\partial \mathbf{h}^{(3)}} \sum_{k=0}^{3} \left(\frac{\partial \mathbf{h}^{(\mathbf{3})}}{\partial \mathbf{h}^{(k)}} \frac{\partial \mathbf{h}^{(k)}}{\partial \mathbf{W}} \right)$$

$$\frac{\partial L}{\partial \mathbf{U}} = \frac{\partial L}{\partial \mathbf{o}} \frac{\partial \mathbf{o}}{\partial \mathbf{h}^{(3)}} \sum_{k=0}^{3} \left(\frac{\partial \mathbf{h}^{(\mathbf{3})}}{\partial \mathbf{h}^{(k)}} \frac{\partial \mathbf{h}^{(k)}}{\partial \mathbf{U}} \right)$$

- Lưu ý: $\frac{\partial L}{\partial W}$ và $\frac{\partial L}{\partial U}$ phụ thuộc vào trạng thái ẩn tại mọi thời điểm

Backprop theo thời gian



In practice, often "truncated" after ~20 timesteps for training efficiency reasons

$$\frac{\partial J^{(t)}}{\partial \boldsymbol{W_h}} = \boxed{\sum_{i=1}^{t} \frac{\partial J^{(t)}}{\partial \boldsymbol{W_h}}\bigg|_{(i)}}$$

Question: How do we calculate this?

Answer: Backpropagate over timesteps $i = t, \ldots, 0$, summing gradients as you go. This algorithm is called **"backpropagation through time"** [Werbos, P.G., 1988, *Neural Networks* **1**, and others]

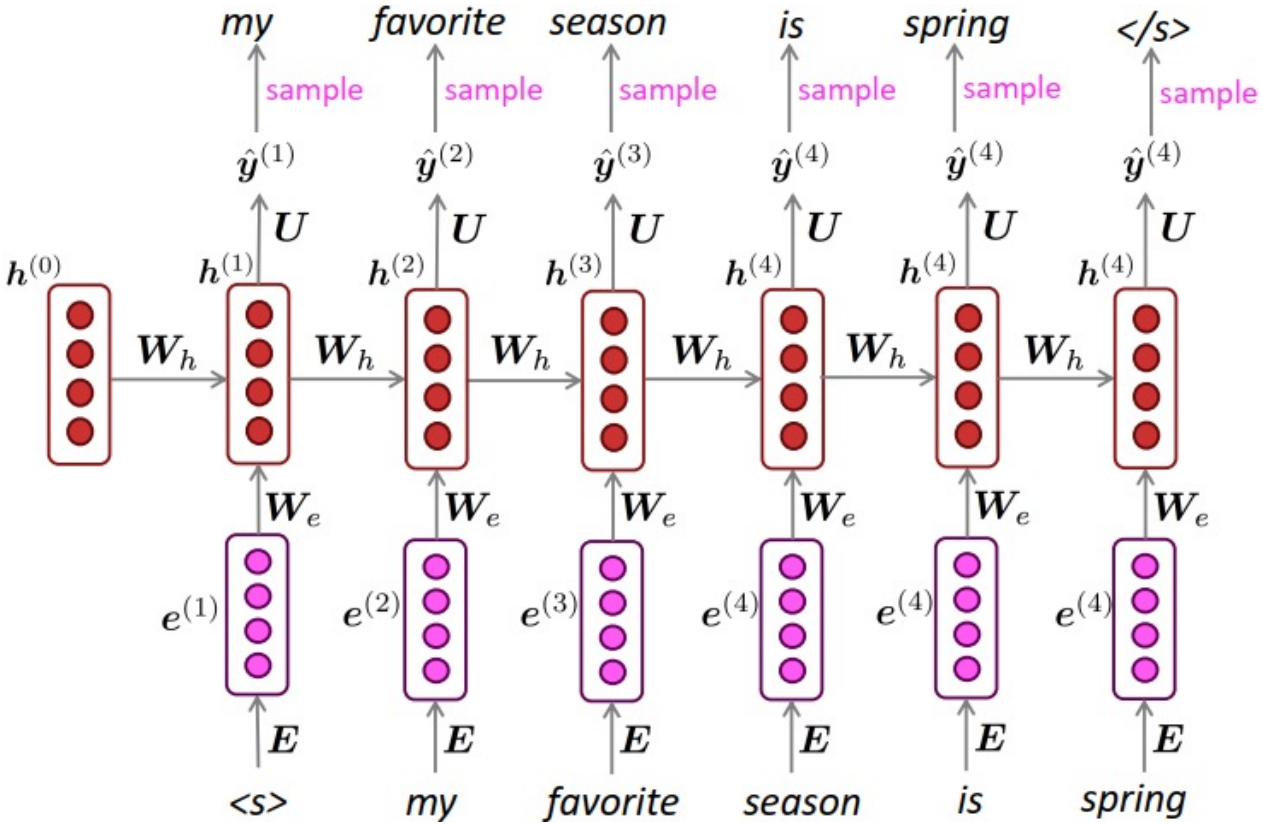
Apply the multivariable chain rule:

$$\frac{\partial J^{(t)}}{\partial \boldsymbol{W}_h} = \sum_{i=1}^{t} \frac{\partial J^{(t)}}{\partial \boldsymbol{W}_h}\bigg|_{(i)} \underbrace{\boxed{\frac{\partial \boldsymbol{W}_h\big|_{(i)}}{\partial \boldsymbol{W}_h}}}_{=1}$$

$$= \sum_{i=1}^{t} \frac{\partial J^{(t)}}{\partial \boldsymbol{W}_h}\bigg|_{(i)}$$

Sinh văn bản với RNN

- Lấy mẫu (sampling) từ tại mỗi bước, đầu ra của bước này sẽ là đầu vào của bước kế tiếp



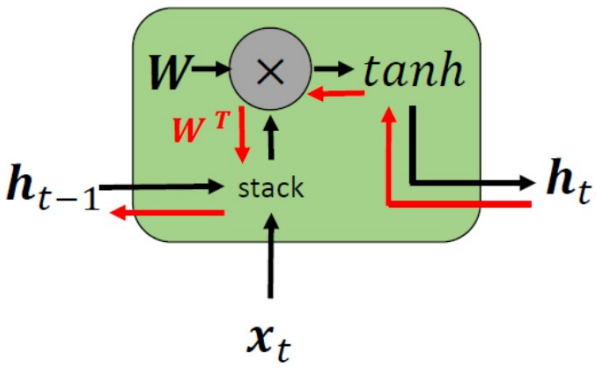
Vanishing và exploding gradient

- Một cách vẽ khác của RNN:

$$h_t = tanh(Wh_{t-1} + Ux_t)$$

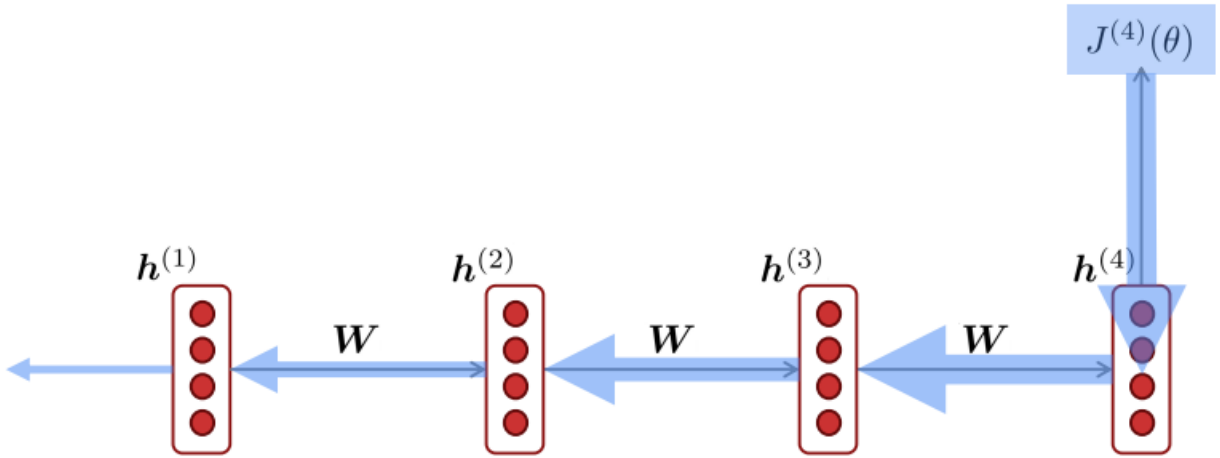
$$\frac{\partial h_t}{\partial W} = (1 - h_t^2)h_{t-1}$$

$$\frac{\partial h_t}{\partial h_{t-1}} = (1 - h_t^2)W^T$$



- Tính gradient của h_0 cần nhân ma trận nhiều lần:
  - Bùng nổ (exploding) gradient: xử lý bằng gradient clipping
  - Tiêu biến (vanishing) gradient: thay đổi cấu trúc RNN

Vanishing gradient

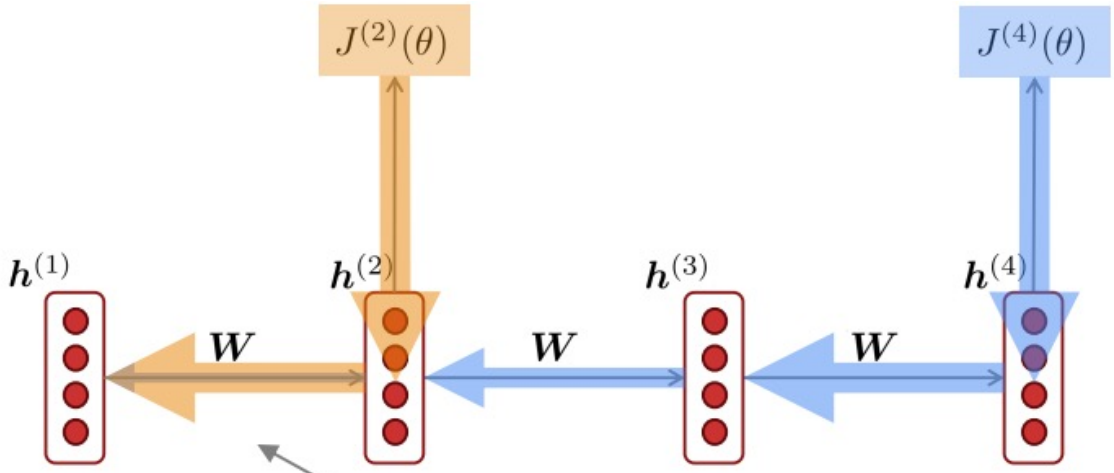


$$\frac{\partial J^{(4)}}{\partial \boldsymbol{h}^{(1)}} = \boxed{\frac{\partial \boldsymbol{h}^{(2)}}{\partial \boldsymbol{h}^{(1)}}} \times \boxed{\frac{\partial \boldsymbol{h}^{(3)}}{\partial \boldsymbol{h}^{(2)}}} \times \boxed{\frac{\partial \boldsymbol{h}^{(4)}}{\partial \boldsymbol{h}^{(3)}}} \times \frac{\partial J^{(4)}}{\partial \boldsymbol{h}^{(4)}}$$

What happens if these are small?

Vanishing gradient problem:
When these are small, the gradient signal gets smaller and smaller as it backpropagates further

Vấn đề với vanishing gradient



Gradient signal from far away is lost because it's much smaller than gradient signal from close-by.

So, model weights are updated only with respect to near effects, not long-term effects.

Giải quyết vanishing gradient

- Vấn đề chính: RNN khó giữ được thông tin qua nhiều bước
- Trong RNN, trạng thái ẩn được cập nhật **liên tục**

$$h^{(t)} = \sigma\left(W_h h^{(t-1)} + W_x x^{(t)} + b\right)$$

- Mô hình RNN với ô nhớ?
  - LSTM
- Tạo kết nối trực tiếp trong mô hình giữa các bước xa nhau
  - Attention, residual connections, etc.

Vấn đề với exploding gradient

- Nếu gradient quá lớn, bước cập nhật trọng số trở nên quá lớn:

$$\theta^{new} = \theta^{old} - \overbrace{\alpha}^{\text{learning rate}} \underbrace{\nabla_\theta J(\theta)}_{\text{gradient}}$$

- Vấn đề: bước nhảy quá lớn dẫn đến trọng số không tốt (giá trị mất mát lớn)
- Trường hợp xấu nhất, mạng nơ ron tồn tại giá trị **Inf** hoặc **NaN** (phải huấn luyện lại mô hình từ bước checkpoint trước)

Gradient clipping: giải pháp cho exploding gradient

- **Gradient clipping**: nếu norm của gradient lớn hơn một ngưỡng định trước, giảm giá trị của gradient trước khi cập nhật trọng số

Algorithm 1 Pseudo-code for norm clipping

$\hat{\mathbf{g}} \leftarrow \frac{\partial \mathcal{E}}{\partial \theta}$
if $\|\hat{\mathbf{g}}\| \geq threshold$ **then**
  $\hat{\mathbf{g}} \leftarrow \frac{threshold}{\|\hat{\mathbf{g}}\|}\hat{\mathbf{g}}$
end if

- **Ý tưởng:** thay đổi trọng số với bước gradient nhỏ hơn
- Trong thực tế, exploding gradient là vấn đề không quá khó để giải quyết